National University of Computer & Emerging Sciences

Computer Science Department

Course Code: CL-2006	**Course:** Operating Systems
Lab Number: 10	**Lab Name:** Process Synchronization using Semaphores

Lab – 10

Process Synchronization using Semaphores (POSIX)

Table of Contents

Objective

In this lab we will study process synchronization using POSIX semaphores in C. We will learn how semaphores work, how to use them to protect shared resources in multi-threaded programs, and how they prevent race conditions. By the end of this lab you will be able to:

- Explain what a semaphore is and how it controls thread access.
- Initialize, wait on, post to, and destroy a semaphore correctly.
- Use binary semaphores as a mutual-exclusion (mutex) mechanism.
- Use counting semaphores to allow limited concurrent access.
- Implement thread-ordering using semaphore signalling.
- Compile and run multi-threaded C programs with -pthread.

Introduction to Process Synchronization

When multiple threads run inside the same process they share the same memory space. This is powerful but dangerous: if two threads read and modify the same variable at the same time, the result is unpredictable. This is called a race condition.

Process synchronization is the set of techniques used to control the order in which threads access shared resources so that correctness is preserved. The core goal is Mutual Exclusion – ensuring that only one thread (or a limited number of threads) is inside the critical section of code at any given time.

> **KEY CONCEPT:** A critical section is any piece of code that reads or writes a shared resource. Only one thread should execute it at a time to prevent data corruption.

There are two main types of locks used in synchronization:

- Shared lock – allows multiple reader threads to access a resource simultaneously (read-only).
- Exclusive lock – allows only one thread to access a resource at a time (read/write).

Semaphores are one of the most fundamental and widely used synchronization tools in operating systems. They are used in everything from device drivers to database engines.

Concept of Semaphores

A semaphore is an integer counter that is used to coordinate access to shared resources across threads or processes. Linux guarantees that operations on a semaphore are atomic – meaning they cannot be interrupted midway.

> **DEFINITION:** A semaphore is a non-negative integer counter that supports only two atomic operations: wait (decrement) and post (increment).

Every semaphore has a value which starts at a number you choose during initialization. The two fundamental operations are:

Wait Operation – sem_wait()

Decrements the semaphore value by 1. If the value is already 0, the calling thread is blocked (put to sleep) until another thread increments it. Think of it as "try to acquire the resource".

Post Operation – sem_post()

Increments the semaphore value by 1. If any threads were blocked waiting, one of them is woken up. Think of it as "release the resource".

There are two types of semaphores:

Named Semaphores

Created with sem_open(). They exist as files in the filesystem (usually under /dev/shm) and can be shared between completely separate processes running from different programs. They persist until explicitly removed with sem_unlink().

Unnamed Semaphores

Created with sem_init(). They live in memory (in a variable you declare) and are typically used within a single process to synchronize threads. They do not exist outside the program and are destroyed when the program exits or when sem_destroy() is called.

> **THIS LAB:** We will use unnamed semaphores (sem_init) throughout all exercises since we are synchronizing threads within a single program.

Implementation of Semaphores

Required Header and Library

To use POSIX semaphores in C you must include the semaphore header and link the pthread library during compilation:

```
#include <semaphore.h>   // Semaphore data types and functions
#include <pthread.h>     // POSIX Threads
#include <stdio.h>
#include <stdlib.h>
```

Compilation command (always use -pthread flag):

```
gcc -o output_name source_file.c -pthread
```

> **IMPORTANT:** Without the -pthread flag, the linker cannot find pthread and semaphore functions and compilation will fail with "undefined reference" errors.

Semaphore Functions Quick Reference

S.NO	Function	Description
1	`sem_init()`	Initializes an unnamed semaphore with a starting value.
2	`sem_destroy()`	Destroys an unnamed semaphore and frees its resources.
3	`sem_wait()`	Decrements (locks) the semaphore. Blocks if value is 0.
4	`sem_post()`	Increments (unlocks) the semaphore. Wakes blocked threads.
5	`sem_getvalue()`	Retrieves the current integer value of the semaphore.

Step 1 – Declaration

A semaphore is represented by the type sem_t. Declare it as a global variable so all threads can access it:

```
sem_t semaphore;          // Declare a semaphore variable
// For multiple semaphores:
sem_t sem1, sem2;
```

Step 2 – Initialization (sem_init)

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Parameters:

- sem – pointer to your sem_t variable.
- pshared – set to 0 for thread sharing (within same process). Set to non-zero for process sharing.
- value – the starting count of the semaphore.

Common initialization patterns:

```
// Binary semaphore (mutex) - only 1 thread at a time:
```

```
sem_init(&semaphore, 0, 1);

// Counting semaphore - up to 3 threads at a time:
sem_init(&semaphore, 0, 3);

// Blocked semaphore - 0 means first wait() blocks immediately:
sem_init(&semaphore, 0, 0);
```

> **TIP:** Initialize with 1 for mutual exclusion (only one thread in critical section). Initialize with 0 for thread ordering (one thread waits until another signals it).

Step 3 – Waiting (sem_wait)

```
int sem_wait(sem_t *sem);
```

sem_wait() atomically decrements the semaphore value by 1. If the value is greater than 0 it decrements and returns immediately. If the value is 0 it blocks the calling thread until another thread calls sem_post().

```
sem_wait(&semaphore);   // Enter critical section
// ... critical section code ...
// Only one thread (or N threads for counting sem) runs this code
```

Step 4 – Posting (sem_post)

```
int sem_post(sem_t *sem);
```

sem_post() atomically increments the semaphore value by 1. If there are threads blocked in sem_wait(), one of them is woken up and allowed to proceed.

```
// ... critical section code ...
sem_post(&semaphore);   // Leave critical section - wake next thread
```

> **RULE:** Every sem_wait() must be matched with exactly one sem_post(). Missing a sem_post() will leave threads blocked forever (deadlock).

Step 5 – Destroying (sem_destroy)

```
int sem_destroy(sem_t *sem);
```

Always call sem_destroy() at the end of your program to release kernel resources associated with the semaphore. Call it only after all threads that use the semaphore have finished.

```
// After all threads have joined:
```

```
sem_destroy(&semaphore);
```

How Semaphores Work – Visual Flow

The diagram below shows how threads interact with a binary semaphore (initial value = 1). Only one thread can hold the semaphore at a time. All others block in queue until it is released.

Figure 1: Semaphore flow – Thread A acquires (sem_wait), Thread B and C block until Thread A releases (sem_post)

Lab Exercises

The following four programs progressively build your understanding of semaphores from basic usage to counting semaphores. Study each program carefully, compile it, run it, and observe the output before moving to the next one.

> **COMPILE ALL PROGRAMS WITH:** gcc -o <output_name> <filename>.c -pthread

Exercise 1 – Basic Semaphore and Thread Ordering

Concept

This exercise introduces the simplest use of a semaphore: controlling the order in which two threads execute. Thread 1 waits on the semaphore (starts blocked because initial value is 0). Thread 2 posts the semaphore, which unblocks Thread 1.

> **KEY POINT:** When sem_init value is 0, the first thread to call sem_wait() will immediately block. It will only proceed when another thread calls sem_post() on the same semaphore.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem;       // Global semaphore
int counter = 0; // Shared variable

void* thread1() {
    sem_wait(&sem);   // Blocks here until sem > 0
    counter++;
    printf("TH1=%d\n", counter);
    sem_post(&sem);   // Signal done
}

void* thread2() {
    sem_post(&sem);   // Unblocks Thread 1
    counter++;
    printf("TH2=%d\n", counter);
    sem_post(&sem);
}

int main() {
    sem_init(&sem, 0, 0);  // Initial value = 0 (Thread 1 will block)
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Code Walkthrough

sem_init(&sem, 0, 0) – initial value is 0, so the semaphore is locked from the start.

Thread 1 calls sem_wait() – value is 0, so Thread 1 immediately blocks and goes to sleep.

Thread 2 calls sem_post() – increments value to 1, which wakes Thread 1.

Thread 1 resumes, increments counter, prints its value, then calls sem_post() itself.

Expected Output

```
TH2=1   (or TH1=1 depending on scheduling)
TH1=2   (Thread 1 always runs AFTER Thread 2 posts)
```

> **OBSERVATION:** The output order of TH1 and TH2 values may vary but Thread 1's increment always happens after Thread 2 posts the semaphore. Notice that counter is NOT protected – both threads modify it without mutual exclusion. This is fixed in Exercise 2.

Exercise 2 – Binary Semaphore as Mutex

Concept

This is the most common real-world use of a semaphore: protecting a shared counter from concurrent modification. With initial value 1, the semaphore acts exactly like a mutex – only one thread can be inside the critical section at a time. All other threads wait in the queue.

> **KEY POINT:** A binary semaphore (initial value = 1) is functionally equivalent to a mutex. sem_wait() = lock, sem_post() = unlock.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define NUM_THREADS   5
#define NUM_INCREMENTS 20

int counter = 0;   // Shared resource
sem_t semaphore;   // Binary semaphore (mutex)

void* increment_counter(void* arg) {
    sem_wait(&semaphore);       // Lock - only one thread proceeds

    for (int i = 0; i < NUM_INCREMENTS; ++i) {
        int temp = counter;
        temp = temp + 1;
        counter = temp;
        printf("Thread %ld -> counter = %d\n", (long)arg, counter);
    }

    sem_post(&semaphore);       // Unlock - next thread proceeds
    return NULL;
}

int main() {
```

```
    pthread_t threads[NUM_THREADS];
    sem_init(&semaphore, 0, 1); // Initial value 1 = binary semaphore

    for (long i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment_counter, (void*)i);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    sem_destroy(&semaphore);
    printf("Final counter: %d\n", counter);
    return 0;
}
```

Code Walkthrough

5 threads are created, all trying to increment a shared counter 20 times each.

Without a semaphore, all 5 threads would interleave and produce a wrong final value due to race conditions.

With sem_init(&semaphore, 0, 1) – value starts at 1, so the first thread to call sem_wait() succeeds immediately. The other 4 block.

The winning thread runs all 20 increments exclusively, then calls sem_post(). The next thread is released, and so on.

Final counter value will always be exactly 5 × 20 = 100 – guaranteed correct.

Expected Output (last few lines)

```
Thread 0 -> counter = 20
Thread 1 -> counter = 40
Thread 2 -> counter = 60
Thread 3 -> counter = 80
Thread 4 -> counter = 100
Final counter: 100
```

> **WHY 100 IS GUARANTEED:** Because each thread finishes ALL its increments before the next thread starts (due to the semaphore being inside the loop). Without the semaphore, the final value would be random and almost certainly wrong.

Exercise 3 – Thread Ordering with Two Semaphores

Concept

This exercise uses two semaphores to enforce strict execution ordering between threads. sem2 starts at 1 (Thread 2 runs first), sem1 starts at 0 (Thread 1 is blocked until Thread 2 signals it). This is a classic producer-consumer pattern in its simplest form.

> **KEY POINT:** Two semaphores can chain threads together into a guaranteed sequence: Thread 2 always runs before Thread 1, regardless of OS scheduling.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <pthread.h>

sem_t sem1, sem2;   // Two semaphores for ordering
int counter = 0;

void* thread1() {
    sem_wait(&sem1);         // Blocked until Thread 2 posts sem1
    counter++;
    printf("TH1=%d\n", counter);
    sem_post(&sem1);
    return NULL;
}

void* thread2() {
    sem_wait(&sem2);         // sem2=1, so proceeds immediately
    counter++;
    printf("TH2=%d\n", counter);
    sem_post(&sem2);         // Release sem2
    sem_post(&sem1);         // Signal Thread 1 to proceed
    return NULL;
}

int main() {
    sem_init(&sem1, 0, 0);   // Thread 1 blocked initially
    sem_init(&sem2, 0, 1);   // Thread 2 starts immediately

    pthread_t t1, t2;
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    sem_destroy(&sem1);
    sem_destroy(&sem2);
    return 0;
}
```

Code Walkthrough

sem1 = 0 → Thread 1 will block immediately at sem_wait(&sem1).

sem2 = 1 → Thread 2 proceeds immediately past sem_wait(&sem2).

Thread 2 increments counter (counter = 1), prints "TH2=1".

Thread 2 calls sem_post(&sem1) – this wakes Thread 1.

Thread 1 increments counter (counter = 2), prints "TH1=2".

Expected Output (always in this order)

```
TH2=1
TH1=2
```

> **GUARANTEED ORDER:** Unlike Exercise 1, this output is deterministic. Thread 2 ALWAYS prints before Thread 1 because Thread 1 cannot proceed until Thread 2 explicitly posts sem1.

Exercise 4 – Counting Semaphore

Concept

A counting semaphore allows more than one thread into the critical section simultaneously – up to the initial count. This is ideal for modeling limited resources such as a pool of 3 database connections, 3 download slots, or 3 printers. Here, up to 3 out of 5 threads can access the resource at the same time.

> **KEY POINT:** Counting semaphore with initial value N means "at most N threads can be in the critical section simultaneously". When the Nth thread enters, the next caller blocks until one exits.

Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_THREADS 5

sem_t semaphore;   // Counting semaphore

void* access_resource(void* arg) {
    int thread_id = (int)(long)arg;

    sem_wait(&semaphore);   // Decrement count; block if count == 0

    // Up to 3 threads can be here at the same time
    printf("Thread %d is accessing the resource\n", thread_id);
    sleep(1);               // Simulate resource usage
    printf("Thread %d is leaving the resource\n", thread_id);

    sem_post(&semaphore);   // Increment count; wake a waiting thread
    return NULL;
}

int main() {
```

```
    pthread_t threads[NUM_THREADS];
    sem_init(&semaphore, 0, 3);  // Allow up to 3 threads simultaneously

    for (long i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, access_resource, (void*)i);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    sem_destroy(&semaphore);
    return 0;
}
```

Code Walkthrough

sem_init(&semaphore, 0, 3) – counting semaphore starts at 3.

Thread 0 calls sem_wait() – value goes from 3 to 2. Thread 0 proceeds.

Thread 1 calls sem_wait() – value goes from 2 to 1. Thread 1 proceeds.

Thread 2 calls sem_wait() – value goes from 1 to 0. Thread 2 proceeds.

Thread 3 calls sem_wait() – value is 0, Thread 3 BLOCKS.

Thread 4 calls sem_wait() – value is 0, Thread 4 BLOCKS.

When Thread 0 finishes and calls sem_post() – value goes to 1, Thread 3 is woken.

Expected Output (order may vary but pattern is consistent)

```
Thread 0 is accessing the resource
Thread 1 is accessing the resource
Thread 2 is accessing the resource
// Thread 3 and 4 wait here until one of the above finishes...
Thread 0 is leaving the resource
Thread 3 is accessing the resource
...
```

> **TIP:** Try changing sem_init to value 1 – you will see only one thread accesses the resource at a time (binary mode). Change to 5 – all threads run simultaneously (no restriction). This shows how counting semaphores are a generalization of binary semaphores.

Semaphore Types – Comparison

The table below summarizes when to use each type of semaphore:

Type	Init Value	Behaviour	Use Case

Binary	1	Only 1 thread at a time	Protecting shared variables
Counting	N (e.g. 3)	Up to N threads at once	Limited resource pools
Signal/Order	0	Thread blocks until signalled	Thread ordering/chaining

Study Problem – The Sleeping Barber

The Sleeping Barber is a classic synchronization problem that demonstrates how semaphores coordinate multiple threads with limited shared resources. Understanding it solidifies your grasp of all the concepts covered in this lab.

Problem Statement

There is a barber shop with:

- 1 barber who cuts hair one customer at a time.
- A waiting room with a limited number of chairs (e.g. 5).
- Customers who arrive at random times.

Rules:

1. If the barber is free and a customer arrives – the customer sits in the barber chair and gets a haircut.
2. If the barber is busy and chairs are available – the customer waits.
3. If all chairs are full – the customer leaves (no space).
4. If no customers are waiting – the barber sleeps.

Semaphore Solution

Three semaphores are used:

- waitingRoom – initialized to number of chairs. Controls how many customers can enter the waiting room. A customer sem_wait()s this before sitting – if 0, they leave.
- barberChair – initialized to 1. Ensures only one customer sits in the barber chair at a time (mutual exclusion on the chair).
- barberPillow – initialized to 0. Used to wake the sleeping barber. A customer sem_post()s this when they sit; the barber sem_wait()s it.

Figure 2: Sleeping Barber problem – semaphore interactions between customer threads and the barber thread

Common Mistakes to Avoid

1. Forgetting -pthread flag

```
gcc myfile.c -o myfile        // WRONG: linker errors
gcc myfile.c -o myfile -pthread  // CORRECT
```

2. Missing sem_post() – Causes Deadlock

If a thread calls sem_wait() but never calls sem_post() (e.g. due to an early return or an exception), all other waiting threads will be stuck forever. This is a deadlock.

3. Using Semaphore Before sem_init()

Always call sem_init() in main() before creating any threads. Using an uninitialized semaphore causes undefined behaviour.

4. Forgetting sem_destroy()

Always call sem_destroy() at the end of main() after all threads have joined. Not destroying semaphores leaks kernel resources.

5. Modifying Shared Data Outside Critical Section

If you read or write a shared variable anywhere outside the sem_wait() / sem_post() block, race conditions can still occur. The entire read-modify-write sequence must be inside the protected section.

Quick Reference Card

```
/* FULL SEMAPHORE LIFECYCLE */

// 1. Include headers
#include <semaphore.h>
#include <pthread.h>

// 2. Declare (global)
sem_t sem;

// 3. Initialize (in main, before threads)
sem_init(&sem, 0, 1);   // 1 = binary/mutex, N = counting, 0 = ordering

// 4. Lock (in thread function)
sem_wait(&sem);
  // ... critical section ...
sem_post(&sem);

// 5. Cleanup (in main, after pthread_join)
sem_destroy(&sem);

// 6. Compile
gcc -o program source.c -pthread
```

Lab Activity

Complete the following tasks during the lab session. Show your output to the instructor before proceeding to the next task.

5. Compile and run Semaphores_1.c. Record the output and explain why Thread 2 always executes before Thread 1.
6. Compile and run Semaphores_2.c. Confirm the final counter is always 100. Now remove the semaphore calls and run again – what happens to the final counter?
7. Compile and run Semaphores_3.c. Confirm the output is always TH2=1, TH1=2. Explain the role of each semaphore.
8. Compile and run Semaphores_4.c. Count how many threads access the resource simultaneously. Change the counting semaphore value to 1 and run again – describe the difference.

9. (Challenge) Modify Semaphores_4.c to also print when a thread starts waiting and when it stops waiting. This will make the blocking behaviour visible in the output.

Conclusion

In this lab you studied POSIX semaphores – one of the most fundamental tools for thread synchronization in C. You learned:

- A semaphore is an atomic counter that controls access to shared resources.
- sem_wait() decrements the count (blocks if 0). sem_post() increments it (wakes blocked threads).
- Binary semaphores (init=1) enforce mutual exclusion, equivalent to a mutex.
- Counting semaphores (init=N) allow up to N threads simultaneously – ideal for limited resource pools.
- Signal semaphores (init=0) enforce thread ordering – one thread waits until another explicitly signals it.
- Always match every sem_wait() with a sem_post(), and always call sem_destroy() at the end.

These building blocks are used in real operating systems, databases, device drivers, and network servers whenever concurrent access to shared resources must be controlled safely.